Из Абрис+ в эксель (для шаблона)

```
# ============================================================

# КОНВЕРТЕР .abr → .xlsx

# Лист «Оглавление» + лист на каждый AbrisPoly.

# Колонки: Тип линии | № начальной | № следующей | Румб | Азимут | Длина

# Азимут СОГЛАСОВАН с румбом (округлён до целых).

# ============================================================


!pip install openpyxl --quiet


import re

import math

from openpyxl import Workbook

from openpyxl.styles import Font, Alignment, Border, Side, PatternFill

from google.colab import files


SCALE_FACTOR = 6.61


# ---------- Утилиты ----------

def find_all_balanced(text, start_type):

    results, pattern = [], re.compile(rf'<Item type="{re.escape(start_type)}
[^"]*"[^>]*>')

    for m in pattern.finditer(text):

        start, i, depth = m.start(), m.end(), 1
```

```
        while i < len(text) and depth > 0:
            no, nc = text.find('<Item', i), text.find('</Item>', i)
            if nc == -1: break
            if no != -1 and no < nc: depth += 1; i = no + 5
            else: depth -= 1; i = nc + 7
        results.append(text[start:i])
    return results


def unescape(s):
    if s is None: return None
    return (s.replace('&amp;#39;', "'").replace('&amp;', '&')
             .replace('&#39;', "'").replace('&quot;', '"'))


def rumb_parts(az_raw):
    """Округляет азимут до целых и возвращает (направление, градусы)."""
    az = az_raw % 360
    rounded = round(az)
    if rounded == 360:
        rounded = 0
    if 0 <= rounded <= 90:
        return ('СВ', rounded)
    elif 90 < rounded <= 180:
        return ('ЮВ', 180 - rounded)
    elif 180 < rounded <= 270:
        return ('ЮЗ', rounded - 180)
```

```
    else:
        return ('СЗ', 360 - rounded)


def rumb_str(direction, deg):
    return f"{direction} {deg}°"


def az_from_rumb(direction, deg):
    """Румб → азимут в градусах (0–360), согласованный с румбом."""
    if direction == 'СВ':
        return deg
    elif direction == 'ЮВ':
        return 180 - deg
    elif direction == 'ЮЗ':
        return 180 + deg
    else:  # СЗ
        return (360 - deg) % 360


def safe_sheet_name(name, used):
    base = re.sub(r'[\\/*?:\[\]]', '_', name or 'Лист')[:31].strip() or 'Лист'
    if base not in used:
        used.add(base)
        return base
    i = 1
    while f"{base[:28]} ({i})" in used:
        i += 1
```

```
    final = f"{base[:28]} ({i})"
    used.add(final)
    return final

def polygon_area_m2(pts_m):
    n = len(pts_m)
    if n < 3:
        return 0
    s = 0
    for i in range(n):
        x1, y1 = pts_m[i]
        x2, y2 = pts_m[(i+1) % n]
        s += x1 * y2 - x2 * y1
    return abs(s) / 2

# ---------- Загрузка ----------
uploaded = files.upload()
filename = list(uploaded.keys())[0]
with open(filename, 'r', encoding='utf-8', errors='replace') as f:
    xml_text = f.read()
print(f"Файл: {filename}, {len(xml_text):,} символов")

# ---------- Точки ----------
points_data = {}
for block in find_all_balanced(xml_text, 'AbrisCore.Objects.AbrisPoint'):
```

```
    idm = re.search(r'<ID[^>]*>([^<]+)</ID>', block)

    if not idm: continue

    pid = idm.group(1)

    loc_match = re.search(r'<Location\s+type="System\.Windows\.Point[^>]*>(.*?)
</Location>',

                          block, re.DOTALL)

    if not loc_match: continue

    lx = re.search(r'<X[^>]*>([-\d.eE+]+)</X>', loc_match.group(1))

    ly = re.search(r'<Y[^>]*>([-\d.eE+]+)</Y>', loc_match.group(1))

    if not (lx and ly): continue

    tail = block.split('<LinkedObjects', 1)[1] if '<LinkedObjects' in block else
''

    tm = re.search(r'<Text[^>]*>([^<]*)</Text>', tail)

    points_data[pid] = {

        'x': float(lx.group(1)),

        'y': float(ly.group(1)),

        'label': unescape(tm.group(1)) if tm else None,

    }

print(f"Точек: {len(points_data)}")



# ---------- Полигоны ----------

polys = []

for block in find_all_balanced(xml_text, 'AbrisCore.Objects.AbrisPoly'):

    guid = re.search(r'<Guid[^>]*>([^<]+)</Guid>', block)

    name = re.search(r'<Name[^>]*>([^<]+)</Name>', block)

    pts_section = re.search(r'<Points>(.*?)</Points>', block, re.DOTALL)
```

```
    if not pts_section or not guid: continue

    polys.append({

        'guid': guid.group(1),

        'name': unescape(name.group(1)) if name else 'Полигон',

        'point_ids': re.findall(r'<PointID[^>]*>([^<]+)</PointID>',
pts_section.group(1)),

    })

print(f"Полигонов: {len(polys)}")



# ---------- Расчёт промеров ----------

def build_rows(poly):

    seen_ids, unique_ids = set(), []

    for pid in poly['point_ids']:

        if pid not in seen_ids and pid in points_data:

            seen_ids.add(pid)

            unique_ids.append(pid)

    n = len(unique_ids)

    if n < 2:

        return []

    rows = []

    for i in range(n - 1):

        a_id, b_id = unique_ids[i], unique_ids[i + 1]

        A, B = points_data[a_id], points_data[b_id]

        dx_m = (B['x'] - A['x']) * SCALE_FACTOR

        dy_m = (B['y'] - A['y']) * SCALE_FACTOR

        dist_m = math.hypot(dx_m, dy_m)
```

```
        az_raw = math.degrees(math.atan2(dx_m, -dy_m)) % 360


        direction, deg = rumb_parts(az_raw)
        rumb = rumb_str(direction, deg)
        az = az_from_rumb(direction, deg)


        rows.append({
            'from': A['label'],
            'to': B['label'],
            'rumb': rumb,
            'az': az,
            'dist': round(dist_m, 0),
            'pt_from': (A['x'], A['y']),
            'pt_to': (B['x'], B['y']),
        })
    return rows


def split_by_last_point(rows):
    if not rows:
        return [], []
    last_label = rows[-1]['to']
    split_idx = None
    for i, r in enumerate(rows):
        if r['to'] == last_label:
            split_idx = i + 1
```

```
            break
    if split_idx is None:
        return [], rows
    return rows[:split_idx], rows[split_idx:]


def contour_area_ha(kontur):
    if len(kontur) < 3:
        return 0.0
    pts_m = []
    for r in kontur:
        pts_m.append((r['pt_from'][0] * SCALE_FACTOR, r['pt_from'][1] *
SCALE_FACTOR))
    last_to = kontur[-1]['pt_to']
    pts_m.append((last_to[0] * SCALE_FACTOR, last_to[1] * SCALE_FACTOR))
    area_m2 = polygon_area_m2(pts_m)
    return area_m2 / 10000


# ---------- Excel ----------
wb = Workbook()
wb.remove(wb.active)
used_names = set()


thin = Side(border_style='thin', color='000000')
border = Border(left=thin, right=thin, top=thin, bottom=thin)
center = Alignment(horizontal='center', vertical='center', wrap_text=True)
```

```
header_font = Font(bold=True, size=11)

header_fill = PatternFill('solid', fgColor='D9E1F2')



toc_rows = []



for poly in polys:

    rows = build_rows(poly)

    if not rows:

        continue

    priv, kontur = split_by_last_point(rows)

    area_ha = round(contour_area_ha(kontur), 2) if kontur else 0.0



    sheet_name = safe_sheet_name(poly['name'], used_names)

    ws = wb.create_sheet(sheet_name)



    headers = [

        'Тип линии',

        'Номер начальной точки (столба) линии',

        'Номер следующей точки (столба) линии',

        'Направление линии, румбы (азимуты), °',

        'Азимут, °',

        'Длина линии, м',

    ]

    for ci, h in enumerate(headers, 1):

        cell = ws.cell(row=1, column=ci, value=h)
```

```
        cell.font = header_font
        cell.alignment = center
        cell.border = border
        cell.fill = header_fill


    current_row = 2
    for r in priv:
        ws.cell(row=current_row, column=1, value='Привязка лесосеки').alignment =
center
        ws.cell(row=current_row, column=2, value=r['from']).alignment = center
        ws.cell(row=current_row, column=3, value=r['to']).alignment = center
        ws.cell(row=current_row, column=4, value=r['rumb']).alignment = center
        ws.cell(row=current_row, column=5, value=r['az']).alignment = center
        ws.cell(row=current_row, column=6, value=r['dist']).alignment = center
        for c in range(1, 7):
            ws.cell(row=current_row, column=c).border = border
        current_row += 1
    for r in kontur:
        ws.cell(row=current_row, column=1, value='Площадь лесосеки').alignment =
center
        ws.cell(row=current_row, column=2, value=r['from']).alignment = center
        ws.cell(row=current_row, column=3, value=r['to']).alignment = center
        ws.cell(row=current_row, column=4, value=r['rumb']).alignment = center
        ws.cell(row=current_row, column=5, value=r['az']).alignment = center
        ws.cell(row=current_row, column=6, value=r['dist']).alignment = center
        for c in range(1, 7):
```

```
            ws.cell(row=current_row, column=c).border = border

        current_row += 1


    ws.column_dimensions['A'].width = 22

    ws.column_dimensions['B'].width = 22

    ws.column_dimensions['C'].width = 22

    ws.column_dimensions['D'].width = 28

    ws.column_dimensions['E'].width = 12

    ws.column_dimensions['F'].width = 14

    ws.freeze_panes = 'A2'


    toc_rows.append({

        'sheet_name': sheet_name,

        'poly_name': poly['name'],

        'points': len(poly['point_ids']),

        'area_ha': area_ha,

    })

    print(f"  Лист '{sheet_name}': привязка {len(priv)}, контур {len(kontur)},
площадь {area_ha} га")


# ---------- Лист «Оглавление» ----------

toc = wb.create_sheet('Оглавление', 0)


toc_headers = ['№', 'Имя листа', 'Тип полигона', 'Точек', 'Площадь, га']

for ci, h in enumerate(toc_headers, 1):

    cell = toc.cell(row=1, column=ci, value=h)
```

```
    cell.font = header_font
    cell.alignment = center
    cell.border = border
    cell.fill = header_fill

for i, row in enumerate(toc_rows, 1):
    r = i + 1
    toc.cell(row=r, column=1, value=i).alignment = center
    toc.cell(row=r, column=2, value=row['sheet_name']).alignment = center
    toc.cell(row=r, column=3, value=row['poly_name']).alignment = center
    toc.cell(row=r, column=4, value=row['points']).alignment = center
    toc.cell(row=r, column=5, value=row['area_ha']).alignment = center
    for c in range(1, 6):
        toc.cell(row=r, column=c).border = border

toc.column_dimensions['A'].width = 6
toc.column_dimensions['B'].width = 28
toc.column_dimensions['C'].width = 22
toc.column_dimensions['D'].width = 10
toc.column_dimensions['E'].width = 14
toc.freeze_panes = 'A2'

# ---------- Сохранение ----------
out = 'lesoseki.xlsx'
wb.save(out)
```

```
print(f"\n✅ Сохранено: {out} (листов: {len(wb.sheetnames)})")

files.download(out)
```